

Recurrent Gap-Time Survival, Rank Tests, and Competing Risks Analysis: `recurSurvTests`

John Muschelli

Abstract

Nonparametric recurrent-event survival curves are available in the statistical literature, but openly available implementations of tests for differences between those curves have been limited. `recurSurvTests` provides R implementations for estimating and comparing recurrent curves on their intended scales: (i) Wang-Chang marginal recurrent gap-time survival and Luo-Huang weighted-risk-set rank tests, (ii) the Peña-Strawderman-Hollander generalized product-limit estimator, (iii) a separately documented Zhao et al. extended-rank implementation with a sensitivity variance path, and (iv) Sivadasan-Sankaran recurrent cause-specific cumulative-incidence functions and their Wald-type comparison. The package is designed to enable tests whose risk sets, weights, and null hypotheses correspond to the curve an investigator has chosen to estimate. Because recurrent gap-time survival, calendar-time event burden, and recurrent competing-risk incidence are distinct estimands, the software makes those choices explicit rather than treating their tests as substitutes. This manuscript documents the methods, data conventions, numerical checks against prior R implementations where available, and reproducible simulation designs. We provide a comparison to published tables and also show an example similar to data analysis from polysomnography sleep data analyzing sleep bouts.

Keywords: recurrent events, gap time, Wang-Chang estimator, weighted risk set, product-limit estimator, competing risks, R

23 Introduction

24 Recurrent events occur when a unit can experience an event repeatedly during observation:
25 hospital admissions, equipment failures, sleep-stage bouts, and recurrent infections are exam-
26 ples. A frequent implementation problem is that a method described as a “recurrent survival
27 curve” may not estimate the same quantity as another method with a similar name. This
28 distinction is especially important when the number of observed recurrences is associated
29 with a participant’s propensity for short gaps.

30 This distinction creates a practical inferential problem, not merely a naming problem. An
31 investigator may plot a recurrent gap-time survival curve and want to test whether that
32 curve differs by treatment, diagnosis, or visit. A test for a calendar-time mean cumulative
33 function, a regression coefficient, or a first-event survival curve instead answers a different
34 question. Thus, the estimand used for the curve and the estimand targeted by its comparison
35 must be aligned before a reported p-value has its intended interpretation.

36 Substantial software already supports recurrent-event regression. For example, the Andersen-
37 Gill [1], Prentice-Williams-Peterson (PWP) [2], and related Cox-model formulations pro-
38 vide score, Wald, and likelihood-based inference for regression coefficients; mixed-effects and
39 frailty extensions are available in `coxme` [3] and `frailtypack` [4]. These are important tools
40 when a conditional covariate or frailty effect is the scientific target. They do not, however,
41 supply a nonparametric global test for equality of an estimated recurrent gap-time survival
42 curve or tests contrasting such survival curves across groups. Existing recurrent-curve R
43 software has also made important contributions: `survrec` [5] includes Wang-Chang and
44 Peña-Strawderman-Hollander estimators, and `newTestSurvRec` [6] includes recurrent-event
45 curve estimators and rank-test procedures. Their available estimators and testing paths do
46 not all target the same curve, so a test cannot be transferred between them solely on the
47 basis of a shared label such as “recurrent survival.”

48 `recurSurvTests` fills the resulting open-source implementation gap by enabling tests that
49 correspond to the recurrent curves under analysis. It is not a general recurrent-event model-
50 ing framework. Its primary aim is to let users estimate a clearly defined recurrent curve and
51 conduct a comparison whose risk set, weighting, and null hypothesis match that curve—in

particular, the Wang-Chang marginal gap-time survival curve and its Luo-Huang weighted-risk- set comparison. The package also keeps alternative estimands explicit, rather than presenting their tests as interchangeable.

Relationship to `newTestSurvRec::Dif.Surv.Rec()`

The existence of `newTestSurvRec::Dif.Surv.Rec()` does not make the Zhao and Luo-Huang implementations redundant, but the functions should not be treated as interchangeable. A source-level inspection of `Dif.Surv.Rec()` shows the following distinctions:

		Zhao et al. (2020) / <code>newTestSurvRec::Dif.Surv.Rec()</code>
Component	<code>newTestSurvRec::Dif.Surv.Rec()</code>	<code>zhao_rank_test()</code>
Curve/risk-set path	The function calls <code>survfitr(..., type = "pe")</code> , i.e. its displayed test path is built from the Peña-Strawderman-Hollander/GPLE-style pooled gap-record fit. The package also contains a separate Wang-Chang fitter, but that is not the estimator selected inside this function.	Uses Wang-Chang weighted risk and event masses, with each subject's eligible gaps scaled by $1/m_i^*$.
Score numerator	At each pooled event time, group-1 events minus the pooled-risk expected group-1 count.	The same broad observed-minus-expected rank-test idea, but with the Wang-Chang weighted counts and the score in Equation 3.

Zhao et al. (2020) / Zhao et al. (2020)		
Component	<code>newTestSurvRec::Dif.Surv.Rec.rank_test()</code>	<code>newTestSurvRec::Dif.Surv.Rec.rank_test()</code>
Weight families	Provides a broad family (<code>LRrec</code> , <code>Grec</code> , <code>TWrec</code> , <code>PPrec</code> , <code>PMrec</code> , <code>PPrrec</code> , <code>HFrec</code> , <code>CMrec</code> , <code>Mrec</code>). For example, <code>Grec</code> uses the total risk-set size and <code>TWrec</code> its square root.	Implements the three paper weights LR, GB, and PP. In the paper’s notation these are 1, $n/Y^*(t)$, and the pooled Wang-Chang survival estimate $S^*(t)$, respectively. Thus the label “GB” is not evidence that the <code>Grec</code> calculation is the same test.
Variance	Uses an event-time variance term based on pooled and group risk-set/event counts, then sums weighted contributions. It does not use Zhao’s subject residual vector.	Uses subject-level residuals from Equation 6 and the robust variance $\widehat{V}^* = \sum_i \hat{\epsilon}_i^{*2}$ in Equation 7. The pooled-risk and group-specific Equation 6 conventions are retained separately here.
Output interpretation	A family of recurrent-curve tests whose default curve path is PSH/GPLE-style.	A test of equality of Wang-Chang marginal recurrent gap-time survival distributions, with a paper-specific robust variance construction.

59 Consequently, `Dif.Surv.Rec(type = "LRrec")` is a useful historical recurrent curve rank
 60 test and an important reference comparison, but it is not an exact implementation of Zhao’s
 61 Equation 3–7 construction. In particular, matching the word “log-rank” does not resolve the
 62 curve estimator, event weighting, or within-subject variance choice. The manuscript’s numer-

ical checks therefore compare the curve estimators separately and do not use `Dif.Surv.Rec()` as a validation oracle for `zhao_rank_test()`.

Three scientific targets are kept separate throughout:

1. Marginal recurrent **gap-time survival**, the target of Wang-Chang and the Luo-Huang rank test.
2. Mean cumulative recurrent-event burden on the **calendar-time** scale, which is not implemented here.
3. Recurrent cause-specific cumulative incidence for **gap times**, the target of the Sivadasan-Sankaran estimators.

This separation avoids calling all recurrent-event comparisons “log-rank tests.” In this manuscript, CIF means cumulative-incidence function, and RCIF means recurrent cumulative-incidence function. METRON is the journal *METRON*, where the Sivadasan-Sankaran recurrent-CIF test was published; that procedure is a weighted contrast/Wald-type procedure and is not a log-rank test.

Data convention and software design

All gap-time survival functions accept a long data frame with one row per observed gap. If an `episode` column is supplied, the package sorts rows within subject by that column; without it, the existing within-subject row order is used. Completed gaps have $\delta_{ij} = 1$; the final row is the terminal right-censored gap with $\delta_{ij} = 0$. The observed gap duration is Y_{ij} . An optional `episode` column makes the ordering explicit. This convention permits a participant with no completed recurrence to contribute a single terminal censored gap.

```
library(recurSurvTests)

gaps <- data.frame(
  id = rep(1:4, each = 3),
  episode = rep(1:3, 4),
  gap = c(2, 3, 4, 1, 4, 3, 2, 2, 5, 1, 3, 4),
  status = rep(c(1, 1, 0), 4),
```

```

arm = rep(c("control", "treatment"), each = 6),
cause = c("A", "B", NA, "A", "A", NA, "B", "A", NA, "B", "B", NA)
)

```

Internal validation enforces this final-censoring convention, nonnegative gap durations, and subject-level constancy of group and paired-visit labels. The core helpers turn long data into subject records and form risk/event components at distinct completed gap times. Keeping these components visible is intended to make formula-to-code comparison possible.

Recurrent gap-time survival estimators

Wang-Chang marginal survival

Let $m_i^* = 1$ for a participant with no complete recurrence and otherwise let m_i^* equal the number of completed gaps. The Wang-Chang weighted risk and event increments [7, 8] are

$$R_{WC}(t) = \sum_i \frac{1}{m_i^*} \sum_{j=1}^{m_i^*} I(Y_{ij} \geq t), \quad dN_{WC}(t) = \sum_i \frac{1}{m_i^*} \sum_{j=1}^{m_i^*} I(Y_{ij} = t, \delta_{ij} = 1).$$

`wc_surv()` calculates

$$\widehat{S}_{WC}(t) = \prod_{u \leq t} \left\{ 1 - \frac{dN_{WC}(u)}{R_{WC}(u)} \right\}.$$

The inverse- m_i^* weighting prevents participants with many observed events from receiving proportionally more influence on the marginal gap distribution. The implementation uses the weighted-risk-set representation shown by Luo and Huang [8] to be algebraically equivalent to the estimator of Wang and Chang [7].

PSH and pooled Kaplan-Meier

The Peña-Strawderman-Hollander (PSH) generalized product-limit estimator [9] uses all observed gap records with unit weight:

$$R_{PSH}(t) = \sum_{i,j} I(Y_{ij} \geq t), \quad dN_{PSH}(t) = \sum_{i,j} I(Y_{ij} = t, \delta_{ij} = 1),$$

$$\widehat{S}_{PSH}(t) = \prod_{u \leq t} \left\{ 1 - \frac{dN_{PSH}(u)}{R_{PSH}(u)} \right\}.$$

Therefore `psh_surv()` is numerically identical to an ordinary pooled Kaplan-Meier calculation on the same long gap records, including terminal censored gaps. They do not diverge merely because censoring is increased. They can appear to differ only if “KM” refers to a different data construction, for example first gaps only, a calendar-time analysis, different treatment of terminal censoring, or weights. PSH has a renewal/IID gap-model interpretation; that interpretation is distinct from the Wang-Chang marginal estimand.

In practice, PSH is most defensible when the scientific target is the distribution of an exchangeable/IID inter-occurrence time under a renewal model (with independent censoring), or when a pooled gap-record curve is being shown explicitly as a descriptive event-weighted quantity. It is not a general replacement for Wang-Chang: subject frailty, dependence of later gaps on earlier gaps, or informative event counts causes participants with more events to contribute more gap records and changes the target. In those settings, `wc_surv()` is preferable for a marginal gap-time survival target, with subject-level bootstrap inference. PSH can still be reported as a sensitivity curve, but it should be labeled as the pooled/renewal estimand rather than as a generic recurrent Kaplan-Meier estimate. The ordinary `survival::survfit()` curve on the same long records is numerically the same pooled product-limit calculation; PSH’s value is the recurrent-event interpretation and its explicit renewal assumptions, not a different product-limit formula.

The legacy PSH implementations return an independence-style no-ties Greenwood/Nelson-Aalen approximation, $\widehat{S}_{PSH}(t)^2 \sum_{u \leq t} dN(u)/R(u)^2$. It is not generally appropriate when within-subject gaps are dependent or event counts are informative. In such settings, clustered uncertainty methods are needed; a point estimator alone does not solve the inference problem.

Subject-bootstrap confidence intervals

`recurrent_surv_ci()` resamples complete participant histories and refits either curve. This is required because recurrent gaps from one participant are dependent, and because the WC weights are functions of the complete history. The `ci_method` argument applies to the recurrent-survival curve CI functions (`recurrent_surv_ci()` and the corresponding curve-contrast workflow): `"bootstrap"` gives subject-bootstrap percentile intervals or max- t bands, whereas `"normal"` gives the legacy independence-style pointwise approximation. The default percentile intervals are pointwise: at a pre-specified time t , they target $P\{S(t) \in CI(t)\} = 1 - \alpha$. They are the conventional uncertainty summary for a plotted curve, but do not support a joint claim about every point of the curve. For that purpose, `interval = "simultaneous"` uses a subject-bootstrap max- t critical value to form a band over an explicitly supplied finite time grid. The associated coverage simulation estimates pointwise and joint coverage separately; its results remain pending until the configured run has completed.

The default number of bootstrap draws is $B = 999$, consistent with common bootstrap and permutation practice. It is adequate for routine exploration, but a two-sided 95% percentile interval estimates each 2.5% tail from only about 25 draws. For final 95% percentile limits or max- t critical values, the documentation recommends $B \geq 2000$ when computation permits, yielding about 50 draws in each percentile tail and reducing Monte-Carlo variation.

A pointwise 95% interval has approximately 95% coverage for $S(t_0)$ at one pre-specified gap time t_0 . Drawing such intervals at multiple times does not yield 95% joint coverage of the full curve. The simultaneous max- t band instead targets $P\{S(t) \text{ is covered for every } t \in \mathcal{T}\} \approx 0.95$ for the user-supplied finite grid $\mathcal{T}$; it is correspondingly wider. Neither output is a confidence interval for a survival quantile (an inverse-curve parameter such as a median gap) or for a between-group curve difference. Those are separate targets. `recurrent_surv_quantile_ci()` provides a whole-subject bootstrap percentile interval for a survival quantile; it does not invert the independence-style normal curve limits because that inversion does not account for within-subject clustering. A between-group quantile or curve-difference interval requires its own two-group cluster-resampling contrast. `recurrent_surv_contrast_ci()` now supplies the latter for pointwise differences, ratios, or log-ratios and for a finite-grid simultaneous band, resampling within

independent arms or complete paired-visit clusters. `recurrent_surv_rmst_ci()` supplies a clustered bootstrap interval for the restricted mean gap time, which remains defined when the median is not reached.

This interface complements, rather than replaces, prior software. The `survrec` WC and PSH fitters return pointwise standard errors, and its plotting method draws normal-approximation pointwise limits. `survrec::survdiffr()` also performs WC or PSH bootstrap resampling for a *selected survival quantile*, such as a median, and returns a `boot` object for subsequent interval calculation. Thus bootstrap inference was not absent from earlier R work. The standard errors returned by the inspected `newTestSurvRec` WC and PSH fitters are the independence-style Nelson-Aalen/Greenwood approximation,

$$\widehat{SE}\{\widehat{S}(t)\} = \widehat{S}(t) \left\{ \sum_{u \leq t} \frac{dN(u)}{R(u)^2} \right\}^{1/2}.$$

Consequently, those values can be used to form a clipped normal pointwise curve interval, $\widehat{S}(t) \pm z_{1-\alpha/2} \widehat{SE}(t)$; they are neither bootstrap intervals nor simultaneous bands. The same independence-style form, evaluated with this package's own event and risk increments, is available with `ci_method = "normal"` to make the legacy-style capability explicit. It reproduces the PSH standard-error calculation on the same long records, and should not be relied on for dependent recurrent gaps or informative event counts, because the displayed variance does not account for subject-level clustering. In contrast, `ci_method = "bootstrap"` provides a whole-curve subject-bootstrap workflow: percentile intervals for fixed times or a max- t band for a finite grid. The resampling itself is conceptually straightforward; its value here is a uniform analyst-facing tool that makes the resampling unit, estimator, time grid, and coverage claim explicit and consistent across WC and PSH analyses. For a more complex bootstrap procedure (e.g. nested), users must bootstrap the data explicitly and then fit the models.

Comparing curves

The following executable example compares Wang-Chang, PSH, and an independently computed pooled Kaplan-Meier curve from `survival` [10] on the same records. It illustrates the

178 estimands, rather than reproducing a figure from a source paper.

```
wc <- wc_surv(gaps, episode = "episode")
psh <- psh_surv(gaps, episode = "episode")

step_at <- function(fit, grid) vapply(grid, function(t) {
  ii <- which(fit$time <= t)
  if (length(ii)) fit$urv[max(ii)] else 1
}, numeric(1))

grid <- sort(unique(c(0, wc$time, psh$time)))
km <- survival::survfit(survival::Surv(gap, status) ~ 1, data = gaps)
km_curve <- data.frame(time = km$time, surv = km$urv)
stopifnot(max(abs(step_at(psh, grid) - step_at(km_curve, grid))) < 1e-12)
par(mfrow = c(1, 2))
plot(grid, step_at(wc, grid), type = "s", ylim = c(0, 1),
      xlab = "Gap time", ylab = "Survival")
lines(grid, step_at(psh, grid), type = "s", col = "firebrick", lty = 2)
legend("topright", c("Wang-Chang", "PSH / pooled KM"),
      col = c("black", "firebrick"), lty = 1:2, bty = "n")
plot(grid, step_at(wc, grid) - step_at(psh, grid), type = "s",
      xlab = "Gap time", ylab = "Wang-Chang minus PSH/KM")
abline(h = 0, lty = 2)
```

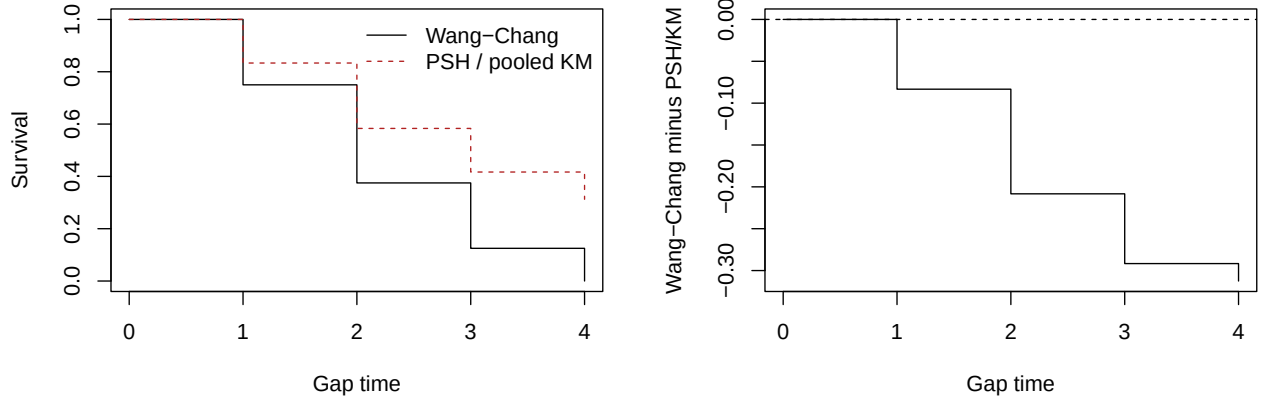


Figure 1: Wang–Chang and PSH/pooled Kaplan–Meier survival on the example records, and their difference.

Rank tests and paired visits

`wc_logrank()` implements the two-group Luo-Huang weighted-risk-set [8] G_ρ^* statistic. At each event time it contrasts weighted observed events with their group-specific weighted risk-set expectation. With $\rho = 0$, it is the recurrent gap-time log-rank analogue. Its implementation uses $w(t) = \widehat{S}_{WC}(t-)^{\rho}$, where the pooled Wang-Chang survival estimate is evaluated just before the event time. Thus $\rho = 1$ downweights late events in proportion to the estimated survival remaining, a Peto-Prentice-type weight; it is not the same as the log-rank weight $w(t) = 1$, which gives every event time equal weight. The variance is built from subject-level contributions to the score: each subject’s observed-minus-expected weighted event increments are summed across that subject’s gaps, preserving their covariance rather than treating the gap rows as independent.

`zhao_rank_test()` is separate. It implements the extended LR, Gehan-Breslow, and Peto-Prentice score construction described by Zhao et al. [11]. Its default `pooled_risk` residual is assessed by the simulations below; current results are diagnostics of this implementation and do not establish byte-for-byte reproduction of the published tables. The group-risk denominator printed in Zhao et al.’s Equation 6 is retained as the explicit `zhao_eq6` sensitivity/reproduction path. Neither function should be described as an exact substitute for the other.

There is an unresolved implementation ambiguity in the published description. Equation

6 appears to define the residual denominator using the risk set of the subject's own group, whereas the pooled-risk construction uses the pooled weighted risk set. These choices produce different subject residuals and hence different robust variance estimates even when the score numerator is unchanged. The package therefore labels them separately (`variance_method = "pooled_risk"` and `"zhao_eq6"`) and does not silently treat one as the exact published convention. We have drafted a clarification request to the authors asking which denominator and finite-sample scaling were used in the published simulations; until that is resolved, the comparisons below are implementation diagnostics rather than a claim of exact reproduction.

For two paired visits per participant, `wc_paired_permutation()` randomly swaps visit labels within participant pairs and recalculates G_ρ^* . Its p-value tests the within-pair label-exchangeability null. The independent-arm analytical p-value from `wc_logrank()` is returned for context but is not a paired analysis. This is the package's permutation-based two-group procedure; the K-group extension currently has an analytical score test (`wc_logrank_k()`), not a general K-group label-permutation wrapper.

Recurrent competing risks

`ss_rcif()` implements recurrent cause-specific cumulative-incidence functions (RCIFs) following Sivadasan and Sankaran [12]. The default shared weighting uses the same subject/event-count denominator for the pooled and cause-specific processes, so the cause-specific increments add to the pooled increment. The 2023 paper's printed Equation 15 instead uses a cause-specific m_{il} denominator and an indicator for subjects with at least two events of that cause. Taken literally, that expression can make the cause-specific increments nonadditive. The `cause_weighting = "literal_eq15"` option is retained for sensitivity and formula-reproduction purposes; it is not the default because additivity is then not guaranteed.

`ss_rcif_equal_causes_test()` is a subject-bootstrap Wald reconstruction of the equal-cause RCIF test proposed by Sivadasan and Sankaran in *METRON* (the journal) [13]. It compares integrated contrasts $F_l(t) - F(t)/k$ across causes. It is a weighted cumulative-incidence contrast with a bootstrap covariance estimate, not a log-rank test, and its reconstruction should not be presented as byte-for-byte author software.

Validation against prior implementations

Each row compares the same event-time grid and survival values with the indicated installed package [5, 6].

```
reference_checks <- manuscript_reference_checks()
reference_display <- reference_checks
reference_display$Max_absolute_difference <- ifelse(
  is.na(reference_checks$Max_absolute_difference), "Not run",
  formatC(reference_checks$Max_absolute_difference, format = "e", digits = 3))
knitr::kable(reference_display)
```

Table 1: Numerical comparisons for Wang–Chang and PSH estimators. Missing differences indicate an unavailable reference package.

Scenario	Estimator	Reference	Version	Max_absolute_difference
100 subjects, 1-5 events	WC	survrec	1.2.5	1.055e-15
100 subjects, 1-5 events	WC	newTestSurvRec	1.0.2	3.333e-09
100 subjects, 1-5 events	PSH	survrec	1.2.5	0.000e+00
100 subjects, 1-5 events	PSH	newTestSurvRec	1.0.2	0.000e+00
80 subjects, 0-6 events	WC	survrec	1.2.5	5.551e-17
80 subjects, 0-6 events	WC	newTestSurvRec	1.0.2	4.999e-09
80 subjects, 0-6 events	PSH	survrec	1.2.5	0.000e+00
80 subjects, 0-6 events	PSH	newTestSurvRec	1.0.2	0.000e+00

These are software comparisons for the estimators of Wang and Chang [7] and Peña et al. [9]. They are not reproductions of the simulation tables in those papers. In particular, newTestSurvRec’s LRrec pathway is associated with PSH-style estimation and is not a numerical reference for the Luo-Huang rank test.

Published-benchmark placeholder for Wang-Chang and Luo-Huang. The curve in Figure 1, the independent pooled-KM check, and Table 1 provide reproducible estimator checks. Direct numerical reproduction of the original Wang-Chang and Luo-Huang simula-

tion tables [7, 8] remains a separate validation task requiring a matched design and transcribed published benchmarks. Zhao’s results below assess a different rank-test construction and cannot fill that role. The existing K-group polysomnography (PSG) simulation is an application-specific extension check, not a reproduction of Luo-Huang’s two-group study.

Simulation validation

Zhao published-table comparison

This section compares completed runs with the “**New method**” columns of Tables 1–3 in Zhao et al. [11]. Table 1 reports the SD of the standardized null statistic; Table 2 reports one-sided type-I error at $\alpha = 0.025$; Table 3 reports power. LR, PP, and GB denote log-rank, Peto-Prentice, and Gehan-Breslow weights. OLR and Jung-Jeong columns are not used as benchmarks for this implementation because the manuscript evaluates the three Zhao extended tests, not the ordinary unadjusted log-rank or the separate Jung-Jeong clustered statistic. Those columns remain useful contextual comparators in the source paper but would answer a different validation question; they are not being claimed as reproduced by `zhao_rank_test()`.

`data-raw/zhao-2020-table-simulations.R` defines 120 cells with four baseline distributions, three heterogeneity ranges, 100 subjects per arm, study end 180 days, and entry uniform on $(0, 180)$. Subject gap multipliers follow $Z \sim U(0.5, 1.5)$, $U(0.1, 1.9)$, or $U(0.01, 1.99)$, labeled low, medium, and high heterogeneity. The intended numbers of simulated datasets are 100,000 per null cell and 10,000 per power cell. Treatment gaps in Table 3 are multiplied by $\exp(0.25)$ or $\exp(0.5)$; for Weibull shape 2 these correspond to changes of 0.5 and 1 in $-\log(\lambda)$.

The baseline in the corrected generator match Table 1: exponential rate $\exp(-4)$, Weibull shape 2 and scale $\exp(4)$, log-normal parameters $(\mu, \sigma) = (4, 0.5)$, and log-logistic survival $S(t) = \{1 + [\exp(-4)t]^2\}^{-1}$. The study-specific generator uses administrative censoring. Zhao used `survsim::rec.ev.surv()` [14] and discussed an additional noninformative censoring mechanism without full parameter details in Section 3. Thus matching baseline distributions does not establish identical simulation designs.

265 The apparent Weibull parameter difference is only a parameterization issue: Zhao’s Table
 266 1 writes $S(t) = \exp(-\lambda t^p)$ with $p = 2$ and $\lambda = \exp(-8)$. Since an R Weibull distribution
 267 uses $S(t) = \exp\{-(t/\text{scale})^p\}$, the corresponding scale is $\lambda^{-1/p} = \exp(4)$, exactly the value
 268 used by the generator. The exponential implementation uses rate $\exp(-4)$, so its mean
 269 is $\exp(4)$; the log-normal uses `meanlog = 4`, and the corrected log-logistic inverse draw
 270 uses scale $\exp(4)$ and shape parameter 0.5. A 200,000-draw source-level sanity check gives
 271 means approximately 54.6, 48.4, 61.9, and 85.5 days for the four distributions, respectively,
 272 consistent with the paper’s stated approximate means 55, 48, 62, and 86. The remaining
 273 differences from Zhao’s published simulations are therefore design differences (notably the
 274 `survsim` generator and incompletely specified censoring mechanism), not an unintended
 275 `exp(4)/exp(-4)` scale inversion.

276 The reference CSV, `data-raw/zhao-2020-published-tables.csv`, was transcribed from
 277 pages 93–95 of the publisher PDF. **Table 2 has a scale ambiguity:** its caption says
 278 “(*100)” but prints integers such as 27 and 25. We interpret these as 0.027 and 0.025,
 279 consistent with Section 3.2’s nominal 0.025 level and interpretation. This is an explicit,
 280 unconfirmed correction; both raw printed values and interpreted probabilities are retained.
 281 Table 2 comparisons are conditional on this interpretation.

```
zhao <- manuscript_zhao(repo_root)
zhao_status <- manuscript_zhao_status(zhao)
zhao_available <- sum(zhao$available)
zhao_comparable <- sum(zhao$available & zhao$design_eligible)
zhao_discrepant <- sum(zhao$comparison == "discrepant")
zhao_method <- if (zhao_available) unique(zhao$variance_method) else "not available"
```

282 At this render, **120 of 120 cells** are available, of which 120 satisfy the recorded design-
 283 eligibility checks. The variance convention recorded in these files is **pooled_risk**. Avail-
 284 ability is assessed from files, not scheduler status. Missing files are shown as *Pending*; they
 285 do not contribute zero rejection rates or enter comparison summaries. `pooled_risk` and
 286 the printed Equation 6 group-risk convention (`zhao_eq6`) remain distinct; the importer re-
 287 jects mixtures of variance conventions and inconsistent settings. The `zhao_eq6` comparison
 288 remains pending unless the separate variance-method array is run and imported from its

isolated result directory.

The cell-availability and design-eligibility counts are retained in the reproducibility artifacts, but are not presented as a scientific table in the manuscript. They are execution metadata rather than results for readers.

Monte Carlo uncertainty and current findings

For a rejection proportion $\hat{p}$ based on B replicates, the Monte Carlo standard error is $\{\hat{p}(1 - \hat{p})/B\}^{1/2}$. Rejection-rate intervals use the Wilson score interval: with $z = z_{1-\alpha/2}$, center $(\hat{p} + z^2/(2B))/(1 + z^2/B)$ and half-width $z\{\hat{p}(1 - \hat{p})/B + z^2/(4B^2)\}^{1/2}/(1 + z^2/B)$. For empirical SDs in Table 1 we use the normal-theory approximation $\text{MCSE}(s) = s/\sqrt{2(B-1)}$; replicate-level fourth moments are unavailable, so this is approximate. A cell receives an asterisk when

$$|\hat{\theta}_{\text{local}} - \hat{\theta}_{\text{paper}}| > 1.96\sqrt{\text{MCSE}_{\text{local}}^2 + \text{MCSE}_{\text{paper}}^2} + 0.0005.$$

The last term allows half a unit of rounding in the published tables. The calculation assumes independent simulation runs and uses the paper’s replicate counts. It is an exploratory cell-wise discrepancy screen, not an equivalence test or a multiplicity-adjusted global validation rule. A cell without an asterisk is not proven equivalent. Nominal calibration is assessed separately in the exported CSV by exact binomial tests against 0.025, with Holm adjustment across the 36 planned Table 2 cells.

Among the 120 comparable completed cells, **82 are discrepant** under the criterion above.

Across 72 available power cells, present minus published power ranges from **+0.46 to +33.44 percentage points**. These results require investigation of the generator/censoring design and variance convention before claiming published-table reproduction.

Under the stated, unconfirmed Table 2 scale interpretation, **30 of 36** local rejection rates are compatible with Zhao’s printed values, while **13 of 36** differ from nominal one-sided $\alpha = 0.025$ after Holm adjustment. All 12 available LR cells reject more often than 2.5%. Agreement with the published empirical rates and calibration to the nominal level are distinct checks.

The original RDS files retain summary moments, not individual simulated statistics or failed-p-value counts. Consequently, the null histogram in Zhao’s Figure 1 cannot be reconstructed from this batch. No histogram is inferred from a mean and SD. The figure below instead compares the reported quantities directly, while the following three tables retain the distribution, heterogeneity, alternative, and LR/PP/GB arrangement of Zhao’s tables.

```
manuscript_zhao_plot(zhao)
```

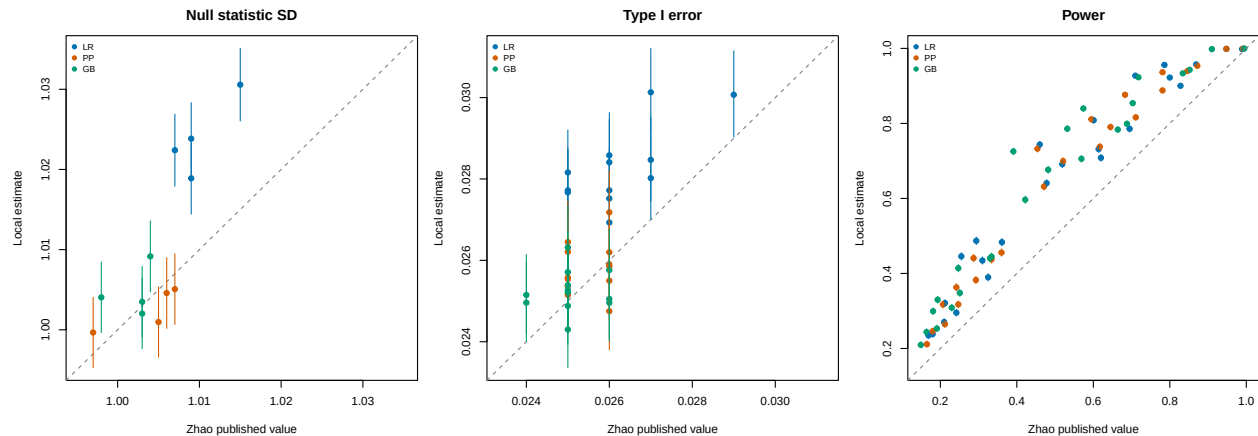


Figure 2: Published versus study simulation summaries. The dashed line is equality. Bars show simulation 95% Monte Carlo intervals (normal approximation for SDs, Wilson intervals for rejection rates). Paper uncertainty and rounding also enter the discrepancy screen. Only available, design-eligible cells are plotted.

```
knitr::kable(manuscript_zhao_table(zhao, "table1"), booktabs = TRUE,
             align = c("l", "l", rep("r", 6)))
```

Table 2: Comparison with Zhao Table 1: empirical SD of the standardized null statistic at medium heterogeneity. LR = log-rank, PP = Peto-Prentice, and GB = Gehan-Breslow. Parentheses contain simulation Monte Carlo SEs; an asterisk marks a discrepancy.

	LR	LR local	PP	PP local	GB	GB local
Distribution	Zhao	(MCSE)	Zhao	(MCSE)	Zhao	(MCSE)
exponential	1.007	1.0224 (0.0023)	1.005	1.0010	1.004	1.0092
		*		(0.0022)		(0.0023)

Table 2: Comparison with Zhao Table 1: empirical SD of the standardized null statistic at medium heterogeneity. LR = log-rank, PP = Peto-Prentice, and GB = Gehan-Breslow. Parentheses contain simulation Monte Carlo SEs; an asterisk marks a discrepancy.

	LR	LR local	PP	PP local	GB	GB local
Distribution	Zhao	(MCSE)	Zhao	(MCSE)	Zhao	(MCSE)
weibull	1.015	1.0306 (0.0023)	1.007	1.0051	1.003	1.0035
		*		(0.0022)		(0.0022)
lognormal	1.009	1.0238 (0.0023)	1.006	1.0046	1.003	1.0020
		*		(0.0022)		(0.0022)
loglogistic	1.009	1.0189 (0.0023)	0.997	0.9997	0.998	1.0040
		*		(0.0022)		(0.0022)

```
knitr::kable(manuscript_zhao_table(zhao, "table2"), booktabs = TRUE,
             align = c("l", "l", rep("r", 6)))
```

Table 3: Comparison with Zhao Table 2: one-sided type-I error probabilities. LR = log-rank, PP = Peto-Prentice, and GB = Gehan-Breslow. Published integers are divided by 1000 as explained in the text. Parentheses contain simulation Monte Carlo SEs; an asterisk marks a discrepancy.

		LR	LR local	PP	PP local	GB	GB local
Heterogeneity	Distribution	Zhao	(MCSE)	Zhao	(MCSE)	Zhao	(MCSE)
low	exponential	0.027	0.0280	0.026	0.0262	0.026	0.0250
			(0.0005)		(0.0005)		(0.0005)
low	weibull	0.029	0.0301	0.026	0.0259	0.025	0.0254
			(0.0005)		(0.0005)		(0.0005)
low	lognormal	0.027	0.0301	0.025	0.0265	0.026	0.0250
			(0.0005) *		(0.0005)		(0.0005)
low	loglogistic	0.026	0.0286	0.026	0.0255	0.025	0.0252
			(0.0005) *		(0.0005)		(0.0005)

Table 3: Comparison with Zhao Table 2: one-sided type-I error probabilities. LR = log-rank, PP = Peto-Prentice, and GB = Gehan-Breslow. Published integers are divided by 1000 as explained in the text. Parentheses contain simulation Monte Carlo SEs; an asterisk marks a discrepancy.

		LR	LR local	PP	PP local	GB	GB local
Heterogeneity	Distribution	Zhao	(MCSE)	Zhao	(MCSE)	Zhao	(MCSE)
medium	exponential	0.026	0.0284 (0.0005) *	0.025	0.0255 (0.0005)	0.026	0.0258 (0.0005)
medium	weibull	0.027	0.0285 (0.0005)	0.026	0.0259 (0.0005)	0.025	0.0263 (0.0005)
medium	lognormal	0.025	0.0277 (0.0005) *	0.025	0.0251 (0.0005)	0.025	0.0257 (0.0005)
medium	loglogistic	0.026	0.0275 (0.0005)	0.025	0.0254 (0.0005)	0.025	0.0243 (0.0005)
high	exponential	0.026	0.0269 (0.0005)	0.026	0.0272 (0.0005)	0.025	0.0249 (0.0005)
high	weibull	0.025	0.0282 (0.0005) *	0.025	0.0256 (0.0005)	0.024	0.0251 (0.0005)
high	lognormal	0.026	0.0277 (0.0005)	0.026	0.0248 (0.0005)	0.025	0.0253 (0.0005)
high	loglogistic	0.025	0.0277 (0.0005) *	0.025	0.0262 (0.0005)	0.024	0.0250 (0.0005)

```
knitr::kable(manuscript_zhao_table(zhao, "table3"), booktabs = TRUE,
             align = c("l", "l", "r", rep("r", 6)))
```

Table 4: Comparison with Zhao Table 3: one-sided power probabilities. LR = log-rank, PP = Peto-Prentice, and GB = Gehan-Breslow. Parentheses contain simulation Monte Carlo SEs; an asterisk marks a discrepancy. The log-time shift is 0.25 or 0.5 for all four distributions.

Heterogeneity	Distribution	Log-time shift	LR	LR local	PP	PP local	GB	GB local
			Zhao	(MCSE)	Zhao	(MCSE)	Zhao	(MCSE)
low	exponential	0.25	0.212	0.3209	0.207	0.3172	0.181	0.2994
				(0.0047) *		(0.0047) *		(0.0046) *
low	exponential	0.50	0.601	0.8085	0.595	0.8111	0.532	0.7858
				(0.0039) *		(0.0039) *		(0.0041) *
low	weibull	0.25	0.460	0.7437	0.454	0.7326	0.391	0.7254
				(0.0044) *		(0.0044) *		(0.0045) *
low	weibull	0.50	0.948	0.9985	0.948	0.9990	0.910	0.9980
				(0.0004) *		(0.0003) *		(0.0004) *
low	lognormal	0.25	0.620	0.7083	0.645	0.7904	0.664	0.7836
				(0.0045) *		(0.0041) *		(0.0041) *
low	lognormal	0.50	0.989	0.9982	0.992	0.9996	0.995	0.9996
				(0.0004) *		(0.0002) *		(0.0002) *
low	loglogistic	0.25	0.325	0.3898	0.334	0.4375	0.334	0.4448
				(0.0049) *		(0.0050) *		(0.0050) *
low	loglogistic	0.50	0.828	0.9003	0.846	0.9396	0.852	0.9430
				(0.0030) *		(0.0024) *		(0.0023) *
medium	exponential	0.25	0.180	0.2384	0.180	0.2462	0.164	0.2438
				(0.0043) *		(0.0043) *		(0.0043) *
medium	exponential	0.50	0.519	0.6914	0.521	0.6996	0.482	0.6765
				(0.0046) *		(0.0046) *		(0.0047) *
medium	weibull	0.25	0.294	0.4871	0.287	0.4410	0.247	0.4140
				(0.0050) *		(0.0050) *		(0.0049) *
medium	weibull	0.50	0.786	0.9560	0.781	0.9365	0.718	0.9230
				(0.0021) *		(0.0024) *		(0.0027) *

Table 4: Comparison with Zhao Table 3: one-sided power probabilities. LR = log-rank, PP = Peto-Prentice, and GB = Gehan-Breslow. Parentheses contain simulation Monte Carlo SEs; an asterisk marks a discrepancy. The log-time shift is 0.25 or 0.5 for all four distributions.

		Log-time	LR	LR local	PP	PP local	GB	GB local
Heterogeneity	Distribution	shift	Zhao	(MCSE)	Zhao	(MCSE)	Zhao	(MCSE)
medium	lognormal	0.25	0.361	0.4834 (0.0050) *	0.360	0.4557 (0.0050) *	0.330	0.4405 (0.0050) *
medium	lognormal	0.50	0.869	0.9573 (0.0020) *	0.872	0.9537 (0.0021) *	0.834	0.9335 (0.0025) *
medium	loglogistic	0.25	0.242	0.2954 (0.0046) *	0.247	0.3174 (0.0047) *	0.230	0.3086 (0.0046) *
medium	loglogistic	0.50	0.695	0.7856 (0.0041) *	0.711	0.8162 (0.0039) *	0.688	0.7989 (0.0040) *
high	exponential	0.25	0.169	0.2347 (0.0042) *	0.165	0.2114 (0.0041) *	0.149	0.2096 (0.0041) *
high	exponential	0.50	0.478	0.6408 (0.0048) *	0.471	0.6316 (0.0048) *	0.422	0.5967 (0.0049) *
high	weibull	0.25	0.255	0.4456 (0.0050) *	0.242	0.3630 (0.0048) *	0.193	0.3300 (0.0047) *
high	weibull	0.50	0.710	0.9273 (0.0026) *	0.683	0.8763 (0.0033) *	0.574	0.8399 (0.0037) *
high	lognormal	0.25	0.310	0.4345 (0.0050) *	0.293	0.3825 (0.0049) *	0.251	0.3481 (0.0048) *
high	lognormal	0.50	0.800	0.9223 (0.0027) *	0.781	0.8880 (0.0032) *	0.703	0.8541 (0.0035) *
high	loglogistic	0.25	0.210	0.2705 (0.0044) *	0.212	0.2647 (0.0044) *	0.191	0.2533 (0.0043) *
high	loglogistic	0.50	0.614	0.7315 (0.0044) *	0.617	0.7378 (0.0044) *	0.569	0.7057 (0.0046) *

The complete pooled-risk run supplies all 120 published-table cells. Rendering uses these stored summaries and does not start simulations. The separate `zhao_eq6` variance convention should be rendered from its own result directory so the two implementations are not pooled. Published-table reproduction remains unresolved because all Table 3 power cells differ from the reported values and the local recurrence/censoring generator is not identical to the paper’s incompletely specified design.

Polysomnography-bout simulation study

Polysomnography (PSG) records sleep in scored 30-second epochs. For a selected sleep stage, consecutive epochs assigned that stage can be run-length encoded into bouts, and the gaps between bouts form recurrent gap times. The supplement simulation is designed to resemble that structure, with: (i) subject frailty, (ii) visit effects, (iii) persistent burst/recovery states, (iv) empirically calibrated 420–800 minute recordings (median 565 minutes) with terminal censoring, (v) unequal independent-arm sizes (7 control, 21 treatment), and (vi) paired visits with shared participant frailty. Under the null, treatment and control gaps have the same distribution. The corresponding PSG-bout data should not be assumed to satisfy the IID gap assumption required for a renewal interpretation of PSH; stage dependence, subject frailty, and bout-state persistence are precisely reasons to prefer the Wang-Chang estimand and clustered inference there. Power alternatives multiply treatment gaps by 1.25 (longer bouts) or 0.80 (shorter bouts). The script evaluates G_0^* , G_1^* , the Zhao pooled-risk implementation, and paired label permutations where appropriate.

The completed study used 10,000 simulated data sets per design–alternative cell and 999 random label swaps for each paired-visit analysis. Thus, under an exactly calibrated 0.05 test, the Monte Carlo standard error of an observed null rejection proportion is about 0.0022; a difference of a few tenths of a percentage point from 0.05 is expected simulation variation, rather than evidence of size distortion. This distinction is important here because the PSG-shaped generator deliberately includes both serial within-visit dependence and substantial subject heterogeneity.

For the paired design, the observed null rejection rates were 0.0516 for the paired G_0^* permutation test and 0.0503 for paired G_1^* . Both are compatible with the nominal 0.05 level

(respectively 0.7 and 0.1 Monte Carlo standard errors above the target). In the independent, unequal-arm design, G_1^* rejected in 0.0481 of null samples, also compatible with nominal size. The corresponding G_0^* rate was 0.0577 (Monte Carlo SE 0.0023; 95% binomial interval 0.0532–0.0625), 3.3 Monte Carlo standard errors above 0.05. This is evidence of modest anti-conservatism for the unweighted statistic in this particular small, 1:3 allocation scenario, not a general claim about its asymptotic size. The default Zhao pooled-risk log-rank sensitivity produced the same rejection proportions as G_0^* in all independent-arm cells, so it does not provide an independent confirmation of calibration in this design. This equality is algebraic, not a coincidental simulation result: at log-rank weight, the Zhao score is $W = \sum_i s_i$, whereas the Luo–Huang statistic is $G_0^* = W/n$. With the pooled-risk denominator, their subject residual is the same $\psi_i = s_i - \sum_j R_{ij} dN_j(Z_i - \bar{Z}_j)/R_j$; consequently their standardized statistics, and hence two-sided p-values, are identical. The **zhao** row is therefore a software cross-check, rather than a separate power comparison. A genuinely distinct Zhao sensitivity analysis must use the printed group-risk (**zhao_eq6**) residual convention.

The power results show useful, but moderate, sensitivity at the available 28-visit scale. For paired visits, rejection probabilities were 0.373 and 0.415 for G_0^* and 0.425 and 0.476 for G_1^* under the 1.25 and 0.80 gap multipliers, respectively. For independent unequal arms, they were 0.171 and 0.093 for G_0^* , versus 0.122 and 0.110 for G_1^* . These directional and weight-specific differences are expected in a non-renewal mixture of short bursts and longer recovery gaps; they should not be read as a universal ranking of the weights. Finally, applying the unpaired Zhao test to paired data was extremely conservative under the null (0.0007) and had rejection rates only 0.0465 and 0.0512 under the alternatives. It is included solely as a warning about ignoring pairing, whereas the paired permutation analyses are the valid comparisons for that design. It is not a permutation test: its visit-level sandwich variance omits the within-pair covariance. Because the two visits share frailty, the relevant paired contrast has a negative covariance term, $-2\text{Cov}(\psi_{i,\text{control}}, \psi_{i,\text{treatment}})$, which reduces its variance when the residuals are positively correlated. Omitting that term overestimates the variance, shrinks the standardized score, and explains the extreme conservatism.

```
psg_file <- Sys.getenv("PSG_RESULTS_FILE", file.path(repo_root, "data-raw/psg-validation"))
if (!grepl("^(/|[A-Za-z]:)", psg_file)) psg_file <- file.path(repo_root, psg_file)
```

```

if (file.exists(psg_file)) {
  psg_object <- readRDS(psg_file)
  # Aggregated files contain both the displayed summary and raw p-values.
  # Accept a summary data frame too, for compatibility with early exports.
  psg_results <- if (is.list(psg_object) && "summary" %in% names(psg_object)) {
    psg_object$summary
  } else {
    psg_object
  }
  stopifnot(is.data.frame(psg_results),
    all(c("design", "alternative", "method", "n_sim", "B", "rejection_rate", "monte_carlo_sd",
    cat("Completed PSG summaries are shown with their actual replicate and permutation counts.
  print(knitr::kable(psg_results, digits = 4))
} else {
  cat("**Results placeholder.** No combined PSG result file is available. Set `PSG_RESULTS` to the path of the result file.
}

```

377 Completed PSG summaries are shown with their actual replicate and permutation counts.

design	alternative	method	rejection_rate	n_sim	monte_carlo_sd	B
paired_visits	longer_bouts	paired_perm_rho0	0.3726	10000	0.0048	999
paired_visits	null	paired_perm_rho0	0.0516	10000	0.0022	999
paired_visits	shorter_bouts	paired_perm_rho0	0.4151	10000	0.0049	999
paired_visits	longer_bouts	paired_perm_rho1	0.4248	10000	0.0049	999
paired_visits	null	paired_perm_rho1	0.0503	10000	0.0022	999
paired_visits	shorter_bouts	paired_perm_rho1	0.4761	10000	0.0050	999
independent_unequal_longer_bouts	longer_bouts	svc_rho0	0.1707	10000	0.0038	999
independent_unequal_null	null	wc_rho0	0.0577	10000	0.0023	999
independent_unequal_shorter_bouts	shorter_bouts	svc_rho0	0.0926	10000	0.0029	999
independent_unequal_longer_bouts	longer_bouts	svc_rho1	0.1221	10000	0.0033	999
independent_unequal_null	null	wc_rho1	0.0481	10000	0.0021	999

design	alternative	method	rejection_rate	n_sim	monte_carlo_se	n
independent_unequal_allocation	shorter_bouts	wc_rho1	0.1100	10000	0.0031	999
independent_unequal_allocation	longer_bouts	zhao	0.1707	10000	0.0038	999
independent_unequal_allocation	null	zhao	0.0577	10000	0.0023	999
independent_unequal_allocation	shorter_bouts	zhao	0.0926	10000	0.0029	999
paired_visits	longer_bouts	zhao_unpaired_sensitivity	0.0465	10000	0.0021	999
paired_visits	null	zhao_unpaired_sensitivity	0.0007	10000	0.0003	999
paired_visits	shorter_bouts	zhao_unpaired_sensitivity	0.0512	10000	0.0022	999

K-group application-specific size and power

The K-group extension was evaluated in 20,000 independent null data sets with 30, 45, and 60 subjects per group and in 20,000 data sets under an ordered alternative (gap multipliers 1.0, 1.2, and 0.8). Under the null, `wc_logrank_k()` rejected 6.775% of samples (exact 95% binomial interval, 6.43%–7.13%), above the nominal 5% level. The interval is narrow enough to rule out Monte Carlo variation as the explanation. Under the ordered alternative, rejection was 88.2% (Monte Carlo SE 0.0023). The null result is a calibration concern for this three-group, unequal-allocation design. The statistic uses a subject-level sandwich covariance with an asymptotic chi-square reference; finite-sample Wald calibration is a plausible source, but the current run does not isolate it from the score/covariance construction or the serial burst/recovery data process. A follow-up varies total sample size and allocation to distinguish these explanations. This is an application-specific extension study, not a published numerical benchmark from Luo and Huang [8].

```
k_file <- file.path(repo_root, "data-raw/wc-logrank-k-validation-results/wc-logrank-k-validation")
if (file.exists(k_file)) {
  k_results <- read.csv(k_file)
  stopifnot(all(c("scenario", "rejection_rate", "mc_se", "n_sim") %in% names(k_results)))
  print(knitr::kable(k_results, digits = 4))
} else {
  cat("**Results placeholder.** No K-group output is available.\n\n")
}
```

}

scenario	rejection_rate	mc_se	n_sim	rejected	ci_low	ci_high	null_p_value
null	0.0678	0.0018	20000	1355	0.0643	0.0713	0
ordered	0.8820	0.0023	20000	17640	0.8774	0.8864	NA

Use of AI-assisted development

AI assistance was used during development to help locate and summarize published methods, draft documentation and simulation scaffolding, translate published formulas into candidate R code, and propose test cases. AI output was not treated as statistical validation. Candidate implementations were reviewed against the cited formulas. The executable checks below assess WC and PSH against independent R packages; these checks do not validate every rank test or competing-risk procedure.

Discussion

The main contribution of `recurSurvTests` is not a claim that one estimator or test fits every recurrent-event question. Rather, it makes the choice visible: PSH/pooled KM for a renewal/IID gap interpretation, Wang-Chang for marginal gap-time survival with event-count weighting, Luo-Huang for the corresponding weighted-risk-set rank comparison, Zhao-style sensitivity analyses, and RCIF methods for recurrent competing risks. The reference checks connect the new implementations to legacy software, while the simulation infrastructure makes remaining empirical questions reproducible rather than implicit.

Future work includes comparing the Zhao pooled-risk and printed Equation 6 variance conventions, resolving the observed published-table power discrepancies, completing the two-group power grid, investigating finite-sample calibration of the K-group extension, and comparing nonparametric tests with marginal and frailty regression models on planned sleep-bout applications.

What must be run before a complete manuscript render

The ordinary examples, estimator/reference checks, table formatting, and manuscript rendering are local and do not launch simulations. The remaining optional outputs have different costs:

Output	Driver	Recommended execution	Input needed by render
Zhao Tables 1–3, pooled risk	<code>data-raw/zhao-2020-table-simulations.R</code> via <code>zhao-2020-table-simulations.sbatch</code>	Complete simulations; report aggregates	<code>data-raw/zhao-2020-table-simulations.R</code> <code>docs/zhao-validation/report.m</code>
Zhao Tables 1–3, printed Equation 6	Same driver with <code>ZHAO_VARIANCE_METHOD=zhao_k</code>	Pending: run the <code>zhao_k</code> HPC array; keep outputs in its separate method directory, then generate the report and variance comparison.	<code>data-raw/zhao-2020-table-simulations.R</code>
PSG-bout study	<code>data-raw/psg-validation-simulation.R</code> via its Slurm array	Complete simulations; 10,000 replicates per cell and 999 paired permutations.	<code>data-raw/psg-validation-results.R</code>
K-group size/power study	<code>data-raw/wc-logrank-k-validation.R</code> via its Slurm array	Complete simulations; 10,000 replicates per scenario; null size was 6.775%, so calibration follow-up is required.	<code>data-raw/wc-logrank-k-validation.R</code>

Output	Driver	Recommended execution	Input needed by render
Two-group power grid	<code>data-raw/wc-logrank-power/</code> via its Slurm array	<code>DesignSimulate.R</code> replicates per cell across N and effect size.	<code>data-raw/wc-logrank-power/</code> .
Curve-interval coverage	<code>data-raw/curve-ci-coverage-reproduction.R</code>	HPFage simulation. 25,000-subject truth plus 1,000 data sets and 999 bootstraps. Verify task partitioning/aggregation before submitting the current array wrapper.	<code>data-raw/curve-ci-coverage-reproduction.R</code> (not currently available).
Zhao report aggregation	<code>data-raw/zhao-2020-validation-report.R</code>	Validation report. completed task outputs and does not simulate.	Existing task RDS files.
Final manuscript artifacts	<code>data-raw/reproduce-manuscript.R</code>	manuscript Quarto and writes public tables plus internal metadata.	Any optional combined outputs supplied above.

The pooled-risk Zhao task directory is complete. The report retains all 120 cells and source checksums; the task RDS summaries do not retain individual simulated statistics. The Equation 6 array is a separate, uncompleted run and must not be mixed with the pooled-risk files.

R version 4.4.0 (2024-04-24)
Platform: x86_64-apple-darwin20

```

421 Running under: macOS 26.6.2
422
423 Matrix products: default
424 BLAS:   /Library/Frameworks/R.framework/Versions/4.4-x86_64/Resources/lib/libRblas.0.dylib
425 LAPACK: /Library/Frameworks/R.framework/Versions/4.4-x86_64/Resources/lib/libRlapack.dylib
426
427 locale:
428 [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
429
430 time zone: America/New_York
431 tzcode source: internal
432
433 attached base packages:
434 [1] stats      graphics  grDevices  utils      datasets  methods   base
435
436 other attached packages:
437 [1] recurSurvTests_0.0.1 testthat_3.3.2      colorout_1.3-0.2
438
439 loaded via a namespace (and not attached):
440 [1] newTestSurvRec_1.0.2 cli_3.6.6           knitr_1.51          rlang_1.2.0
441 [5] xfun_0.57           ote1_0.2.0          pkgload_1.5.2        jsonlite_2.0.0
442 [9] rprojroot_2.1.1     htmltools_0.5.9     survrec_1.2-5        tinytex_0.59
443 [13] pkgbuild_1.4.5      brio_1.1.5           rmarkdown_2.31       grid_4.4.0
444 [17] evaluate_1.0.5      fastmap_1.2.0        yaml_2.3.12          lifecycle_1.0.5
445 [21] compiler_4.4.0      here_1.0.2           rstudioapi_0.18.0    lattice_0.22-6
446 [25] digest_0.6.39       R6_2.6.1             splines_4.4.0        magrittr_2.0.5
447 [29] Matrix_1.7-2        tools_4.4.0          boot_1.3-31          survival_3.8-3
448 [33] desc_1.4.3

```

References

- [1] P. K. Andersen and R. D. Gill. Cox's regression model for counting processes: A large sample study. *The Annals of Statistics*, 10(4):1100–1120, 1982. doi: 10.1214/aos/1176345976.
- [2] R. L. Prentice, B. J. Williams, and A. V. Peterson. On the regression analysis of multivariate failure time data. *Biometrika*, 68(2):373–379, 1981. doi: 10.1093/biomet/68.2.373.
- [3] Terry M. Therneau. *coxme: Mixed Effects Cox Models*, 2024. URL <https://CRAN.R-project.org/package=coxme>. R package version 2.2-22.
- [4] Virginie Rondeau, Yassin Mazroui, and Juan R. Gonzalez. frailtypack: An R package for the analysis of correlated survival data with frailty models using penalized likelihood estimation or parametrical estimation. *Journal of Statistical Software*, 47(4):1–28, 2012. doi: 10.18637/jss.v047.i04.
- [5] Juan R. Gonzalez, Edsel A. Peña, and Robert L. Strawderman. *survrec: Survival Analysis for Recurrent Event Data*, 2018. URL <https://github.com/isglobal-brge/survrec>. R package version 1.2-5.
- [6] Carlos Miguel Martinez Manrique. *newTestSurvRec: Statistical Tests to Compare Curves with Recurrent Events*, 2018. URL <https://CRAN.R-project.org/package=newTestSurvRec>. R package version 1.0.2.
- [7] Mei-Cheng Wang and Shu-Hui Chang. Nonparametric estimation of a recurrent survival function. *Journal of the American Statistical Association*, 94(445):146–153, 1999. doi: 10.1080/01621459.1999.10473831.
- [8] Xianghua Luo and Chiung-Yu Huang. Analysis of recurrent gap time data using the weighted risk-set method and the modified within-cluster resampling method. *Statistics in Medicine*, 30(4):301–311, 2011. doi: 10.1002/sim.4074.
- [9] Edsel A. Peña, Robert L. Strawderman, and Myles Hollander. Nonparametric estimation with recurrent event data. *Journal of the American Statistical Association*, 96(456):1299–1315, 2001. doi: 10.1198/016214501753381887.

- 477 [10] Terry M. Therneau. *A Package for Survival Analysis in R*, 2024. URL [https://CRAN.R-](https://CRAN.R-project.org/package=survival)
478 [project.org/package=survival](https://CRAN.R-project.org/package=survival). R package version 3.8-3.
- 479 [11] Qiang Zhao, Bin Zhang, Michael P. LaValley, Joseph M. Massaro, Kathryn L. Lunetta,
480 and Mark Chang. Extended rank tests for analyzing recurrent event data. *Statistics in*
481 *Biopharmaceutical Research*, 12(1):90–98, 2020. doi: 10.1080/19466315.2019.1601596.
- 482 [12] Sisuma Mandakathingal Sivadasan and Paduthol Godan Sankaran. Nonparametric es-
483 timation of cumulative incidence functions of recurrent events. *Statistica*, 83(1):3–25,
484 2023. doi: 10.6092/issn.1973-2201/11727.
- 485 [13] M. S. Sisuma and P. G. Sankaran. Non-parametric test of recurrent cumulative incidence
486 functions for competing risks models. *METRION*, 80(3):331–342, 2022. doi: 10.1007/s4
487 0300-022-00228-x.
- 488 [14] David Moriña and Albert Navarro. The R package survsim for the simulation of simple
489 and complex survival data. *Journal of Statistical Software*, 59(2):1–20, 2014. doi: 10.1
490 8637/jss.v059.i02.